Day 21: Layered Driver Architecture

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

HAL Layer Design

The **Hardware Abstraction Layer** defines **what** a peripheral can do, not **how** it does it. It's an abstract C++ interface:

```
class ITemperatureSensor {
public:
    virtual ~ITemperatureSensor() = default;
    virtual bool    init()                = 0;
    virtual bool    sample()              = 0;
    virtual int32_t temperature_mdeg() const = 0;  // millidegrees C
};
```

The HAL layer never mentions "BME280" or "nRF52840" — it's hardware-agnostic.

Driver Layer (Device-Specific)

The driver layer **implements** the HAL for a specific piece of hardware:

```
class Bme280Sensor : public ITemperatureSensor {
    bool init()   override { /* configure BME280 via SPI */ return true; }
    bool sample() override { /* read BME280 registers */   return true; }
    int32_t temperature_mdeg() const override { return cached_temp_; }
private:
    const struct device *spi_dev_;
    int32_t cached_temp_;
};
```

The driver knows about the specific chip's register map, SPI protocol, and timing requirements.

Application Layer

The application uses **only the HAL interface** — it never knows which sensor is connected:

```
void run_temperature_monitor(ITemperatureSensor &sensor) {
    sensor.init();
    while (true) {
        sensor.sample();
        int32_t t = sensor.temperature_mdeg();
        if (t > 85000) { /* trigger alert */ }
        k_msleep(1000);
    }
}
// Works with any ITemperatureSensor implementation:
run_temperature_monitor(bme280);   // Real hardware
run_temperature_monitor(fake);     // Unit test double
```

SPI & I2C Zephyr Wrapper Classes

Wrapping Zephyr's SPI/I2C APIs in a class makes driver code cleaner:

```
class SpiDevice {
public:
    bool write(const uint8_t *buf, size_t len);
    bool read(uint8_t *buf, size_t len);
    bool transfer(const uint8_t *tx, uint8_t *rx, size_t len);
private:
    const struct device *bus_;
    struct spi_config    cfg_;
    struct gpio_dt_spec  cs_;
};
```

Separation of Concerns

Each layer has one responsibility:

- **HAL:** defines the contract (what)
- **Driver:** implements the contract for specific hardware (how)
- **Application:** uses the contract to accomplish goals (why)

This separation enables: swapping hardware without touching application code, unit testing with fake drivers, supporting multiple hardware variants with the same application.

Zephyr's Sensor Subsystem

Zephyr's built-in sensor subsystem follows exactly this pattern:

```
// HAL: <zephyr/drivers/sensor.h>
sensor_sample_fetch(dev);
sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &val);
// Driver: drivers/sensor/bme280/bme280.c — implements the sensor API
// Application: your code using sensor_sample_fetch() without knowing it's a BME280
```

Practice Tasks

1. Add a `IHumiditySensor` interface and implement it for the nRF52840 internal humidity sensor (if available) and a fake version
2. Write a `SpiDevice` wrapper class and use it inside a `Bme280` driver — test with a real or simulated BME280
3. Create a unit test (host-side) that uses `FakeSensor` to test your application logic without hardware
4. Implement the same driver for two different sensors (e.g., NTC thermistor + digital sensor) and swap them without changing application code

---

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day21
west build -b nrf52840dk/nrf52840 .
west flash
```

```
# View output via RTT or UART serial (115200 baud)
west attach   # RTT viewer (requires J-Link)
```

Resources

- Zephyr Docs
- nRF52840 Product Specification
- Nordic DevZone
- Source: src/day21/main.cpp